



Adding cqs to Cursor

Step-by-step setup for the Code Quality Studio MCP server

5 minutes

7 tools

no API key

Five minutes. No prior knowledge of MCP needed.

What you're setting up

Cursor's AI agent can normally only read your code and talk about it. **MCP** (Model Context Protocol) lets it run real tools on your machine. After this setup you can type things like *"audit my tests and show me a report"* and the agent does it — running the audit locally, reading the results, explaining them.

Nothing is uploaded. The audit runs on your machine, on your files.

Step 1 — Install cqs

Open a terminal:

```
npm install -g cqs-audit
```

Check it worked:

```
cqs --help
```

You should see help text with a version number. If you get `command not found`, see [Troubleshooting](#).

Requires Node.js 18+. Check with `node --version`.

Step 2 — Let cqs write the config for you

```
cqs mcp-config cursor
```

It prints a ready-to-paste block with the correct paths for *your* machine:

```
{
  "mcpServers": {
    "cqs": {
      "command": "/absolute/path/to/node",
      "args": ["/absolute/path/to/cqs-mcp.js"]
    }
  }
}
```

Copy that. Step 3 is where it goes.

Why absolute paths, and not just "cqs-mcp" or "npx" ? An app launched from the Dock or Finder does not inherit your terminal's `PATH` — on macOS it gets roughly `/usr/bin:/bin:/usr/sbin:/sbin`. If you use `nvm`, everything lives under `~/.nvm`, so `cqs-mcp`, `node` and `npx` are all invisible to Cursor. Absolute paths are the only thing that reliably works, which is why the tool prints them rather than asking you to find them.

If you prefer to find them yourself

```
echo "command: $(which node)"
echo "args:    $(npm root -g)/cqs-audit/dist/cqs-mcp.js"
```

Take the second path from `npm root -g`, not from a folder you happen to remember. If npm has a `prefix` configured, or you use `nvm`, there may be more than one `cqs-audit` on disk and only one of them is the one npm updates.

Check for stale copies if something looks wrong:

```
npm config get prefix                                # where installs land
cat "$(npm root -g)/cqs-audit/package.json" | grep '"version"'
cqs --help | head -2                                  # should agree
```

If those two versions disagree, you have more than one copy and something is reading the wrong one. Delete the one that is *not* under `npm root -g`.

Step 3 — Add it to Cursor's config

Open (or create) `~/.cursor/mcp.json` and add a `cqs` entry:

```
{
  "mcpServers": {
    "cqs": {
      "command": "PASTE_YOUR_COMMAND_PATH_HERE",
      "args": ["PASTE_YOUR_ARGS_PATH_HERE"]
    }
  }
}
```

Filled in, it looks like this:

```
{
  "mcpServers": {
    "cqs": {
      "command": "/Users/you/.nvm/versions/node/v24.20.0/bin/node",
      "args": ["/Users/you/.nvm/versions/node/v24.20.0/lib/node_modules/cqs-audit/dist/cqs-
mcp.js"]
    }
  }
}
```

If the file already has other servers in it, do not replace it. Add `"cqs": { ... }` inside the existing `"mcpServers"` block, with a comma between entries. One misplaced comma makes the JSON invalid and then *none* of your servers load — not just this one.

Check it parses before moving on:

```
python3 -m json.tool ~/.cursor/mcp.json
```

If that prints your config, it's valid. If it prints an error, fix the comma.

Global or per project?

The file above is global — `cqs` appears in every project you open. To scope it to one repository instead, put the same JSON in `.cursor/mcp.json` inside that project.

Be aware that a project file usually gets committed, and it contains absolute paths from *your* machine which will not exist on a teammate's. Global is the safer default.

Step 4 — Restart Cursor properly

Quit Cursor completely — Cmd+Q on a Mac. Closing the window is not enough; the config is only read when the app starts.

Then open **Cursor Settings** → **MCP**. You should see `cqs` listed with a green indicator and **7 tools**.

If it is red, open it — Cursor shows the specific error there. See [Troubleshooting](#).

Step 5 — Use it

Open a project and ask the agent in chat. You never name the tools; the agent picks them.

Ask this	What happens
"Audit the tests in ./tests and give me the five worst problems."	Runs the audit, summarises the worst findings
"Generate a cqs report for ./tests and give me the file path."	Writes a self-contained HTML report you can open and share
"Which of my test files have no assertions?"	Filters to that specific rule
"What rules does cqs check for Playwright reliability?"	Lists the rules in that category
"Is my cqs-rules.json valid?"	Validates your custom rules file

The seven tools

Tool	Purpose
<code>cqs_audit</code>	Find quality problems in a file or folder
<code>cqs_report</code>	Write a full visual HTML report
<code>cqs_list_stacks</code>	List the 18 supported frameworks
<code>cqs_list_rules</code>	List the rules for a framework
<code>cqs_validate_rules</code>	Check a custom rules file
<code>cqs_test_rule</code>	Try a rule before committing it
<code>cqs_read_report</code>	Summarise a saved JSON report

Troubleshooting

command not found: cqs after installing

Global npm packages install per Node version *unless* npm has a prefix set. If you use `nvm` and switch versions, `cqs` may not be there. Check both:

```
node --version
npm config get prefix
npm install -g cqs-audit@latest
```

Cursor reports a different version than my terminal

You have two copies and Cursor is reading the stale one. See [Make sure you copied the path npm actually uses](#).

ERR_MODULE_NOT_FOUND when running cqs

Upgrade: `npm install -g cqs-audit@latest`. Versions before 2.3.3 shipped a bundle with an unresolvable import and fail on every command, including `--help`.

Cursor shows `cqs` in red / "error"

Work through these in order:

1. **Did you fully quit and reopen?** Cmd+Q, not just closing the window.
2. **Is Cursor mid-update?** An interrupted or in-progress update can make

servers fail to start. Let it finish, then relaunch.

1. **Do the paths resolve?** Run these — both must succeed:

```
`` bash ls -l "$(python3 -c "import
json,os;print(json.load(open(os.path.expanduser('~/.cursor/mcp.json'))['mcpServers']['cqs']
['command']))")" ls -l "$(python3 -c "import
json,os;print(json.load(open(os.path.expanduser('~/.cursor/mcp.json'))['mcpServers']['cqs']
['args'][0]))")" ``
```

1. **Is the JSON valid?** `python3 -m json.tool ~/.cursor/mcp.json`
2. **Read Cursor's own error.** Settings → MCP → click the server. It names

the reason, which beats guessing.

Test the server without Cursor

This proves the server works, independently of any editor. It deliberately runs with an empty environment, the same way Cursor launches it:

```
env -i /path/to/node /path/to/cqs-mcp.js <<<
'{"jsonrpc":"2.0","id":1,"method":"initialize","params":{"protocolVersion":"2024-11-
05","capabilities":{},"clientInfo":{"name":"t","version":"1"}}}
{"jsonrpc":"2.0","id":2,"method":"tools/list"}
```

A wall of JSON containing `cqs_audit` and `cqs_report` means the server is healthy and the problem is in Cursor's config or restart — not in cqs.

The agent can see the tools but says it cannot find files

Use paths relative to the open project, or give an absolute path. The server resolves paths from where it was started, which is not always where you think.

Frequently asked

Does my code leave my machine? No. The rules are local pattern matching — no network, no API key, no account.

Do I need an API key? Not for any of this. Keys are only needed for the optional AI review and the fix agents, which are separate features.

Does this slow Cursor down? No. The server is idle until the agent calls it, and an audit of a hundred files takes well under a second.

How do I remove it? Delete the `"cqs"` block from `~/.cursor/mcp.json` and restart Cursor. To uninstall entirely: `npm uninstall -g cqs-audit`.

Does this work in other editors? Yes — the same server works in Claude Desktop, Claude Code and any MCP client. Only the config file location differs.